Sistema de agendamiento y control para un estudio de belleza

Informe de avance — Proyecto "Valeria Guerrero Studio"

Miguel Montenegro
Facultad de Ingeniería
Universidad Nacional de Colombia
Bogotá, Colombia

Jose Angulo
Facultad de Ingeniería
Universidad Nacional de Colombia
Bogotá, Colombia

***Resumen*—Este informe presenta el avance del proyecto final de Programación Orientada a Objetos: un programa en Java para Valeria Guerrero Studio, un emprendimiento de belleza (uñas, cejas y pestañas) atendido por una sola persona. A partir de una entrevista con la dueña se identificaron sus necesidades: agendar citas sin cruces, guardar la información de cada clienta, generar recordatorios para WhatsApp y llevar el control de ingresos y gastos. Se describen los objetivos, los requerimientos levantados, el diagrama de casos de uso depurado a los seis casos esenciales, el diagrama de clases y las cinco clases implementadas (Cliente, TratamientoEstetico, Cita, Pago y Negocio). La prueba en el método `main` muestra que el sistema rechaza citas que se cruzan, calcula la hora de finalización con un margen de 30 minutos, genera mensajes personalizados y consolida el resumen financiero.**

***Palabras clave*—programación orientada a objetos, Java, UML, casos de uso, diagrama de clases, agendamiento de citas.**

I. Introducción

Valeria Guerrero Studio es un emprendimiento de belleza que funciona en una casa, en un espacio privado, y cuya propietaria trabaja sola. Hoy las citas se anotan en un cuaderno o en un calendario, muchas veces de memoria, y los ingresos se registran en una libreta con el valor y el día. En la entrevista, la dueña señaló tres dificultades concretas: no calcular bien cuánto tarda cada servicio para ubicar la siguiente cita, olvidar citas ya programadas y las inasistencias (unas cinco clientas no han llegado).

El proyecto propone un programa de escritorio en Java que lleve la agenda, el perfil de las clientas, los recordatorios y las finanzas del negocio. El sistema se diseñó con Programación Orientada a Objetos (POO): cada concepto real del negocio (clienta, tratamiento, cita, pago) se modela como una clase con atributos privados y métodos públicos, y una clase central, `Negocio`, coordina a las demás. El desarrollo se realiza en Apache NetBeans.

Este documento corresponde al informe de avance. La Sección II presenta los objetivos; la Sección III, los requerimientos de la clienta; la Sección IV, los diagramas de casos de uso y de clases; la Sección V, la explicación de cada clase; la Sección VI, la prueba del programa; la Sección VII, el estado de avance, y la Sección VIII, las conclusiones.

II. Objetivos

A. Objetivo general

Desarrollar un programa en Java, aplicando programación orientada a objetos, que permita a Valeria Guerrero Studio agendar citas, guardar la información de sus clientas y llevar el control de sus ingresos y gastos.

B. Objetivos específicos

1) Diseñar las clases del sistema (`Cliente`, `TratamientoEstetico`, `Cita`, `Pago` y `Negocio`) a partir del diagrama UML y de las necesidades de la dueña del negocio.
2) Implementar el agendamiento de citas, calculando la hora de finalización según los servicios y validando que no se crucen horarios.
3) Registrar el historial de cada clienta: visitas, total gastado y última visita.
4) Generar mensajes de recordatorio para las citas del día y del día siguiente.
5) Registrar los pagos, diferenciando efectivo, Nequi/QR y transferencia, y los gastos, para calcular la ganancia del negocio.

III. Requerimientos de la clienta

A. Lo que solicitó la dueña del negocio

Los requerimientos se obtuvieron mediante una entrevista estructurada con la propietaria [1], técnica habitual de obtención de requerimientos [4], organizada en siete bloques: información del negocio, funcionamiento actual, servicios, clientas, citas y agenda, recordatorios, dinero y uso del programa.

Agenda. Atiende de lunes a sábado entre las 9:00 y las 21:00 y los domingos solo en la mañana, en promedio dos clientas por día. Deja cerca de media hora entre una cita y otra y quiere ver la agenda por semana. Una clienta puede hacerse varios servicios en la misma cita, y algunos servicios requieren retoque: las pestañas, cada 15 a 19 días.

Clientas. Quiere guardar nombre, celular, cumpleaños y cómo conoció el negocio (principalmente TikTok y recomendaciones), además de observaciones como gustos, sensibilidad en piel u ojos o si suele llegar tarde. Al abrir un perfil espera ver los servicios realizados, cuánto ha gastado, la última visita y la frecuencia. También quiere marcar las

TABLA I
REQUERIMIENTOS DEL SISTEMA Y ESTADO DE AVANCE

ID	Requerimiento	Prior.	Estado
RF1	Registrar clientas con nombre, celular, cumpleaños, cómo conoció el negocio y observaciones	A	Hecho
RF2	Consultar el perfil e historial de la clienta (visitas, total gastado, última visita)	A	Hecho
RF3	Agendar citas con uno o varios tratamientos, calcular la hora de fin con 30 min de margen y rechazar cruces	A	Hecho
RF4	Consultar la agenda	A	Parcial[a]
RF5	Generar recordatorios para hoy y mañana con mensaje personalizado	A	Parcial[b]
RF6	Cambiar el estado de la cita: completada, cancelada o no llegó	M	Hecho
RF7	Registrar el pago de una cita diferenciando el método (efectivo, Nequi/QR, transferencia)	M	Hecho
RF8	Registrar gastos del negocio	M	Hecho
RF9	Mostrar resumen financiero: ingresos, gastos, ganancia neta e inasistencias	M	Parcial[c]
RF10	Mantener el catálogo de tratamientos con precio, duración y retoque	M	Hecho
RF11	Sugerir la fecha del próximo retoque	B	Hecho
RNF1	Implementación en Java con POO: atributos privados, constructores, getters y setters	–	Hecho
RNF2	Uso sencillo: una sola usuaria (la propietaria) y mensajes listos para copiar	–	Parcial
RNF3	Respetar el horario de atención y los días bloqueados por compromisos personales	–	Pendiente

[a]Hoy por día; falta la vista semanal. [b]Genera el texto; el envío automático por WhatsApp queda fuera del alcance actual. [c]Faltan servicios más realizados y días más llenos.

inasistencias "para conocer quiénes son".

Recordatorios. Sus clientas le escriben por WhatsApp. Pidió recordatorios un día antes y el mismo día de la cita, con un mensaje "agradable y femenino" que empiece con "Hola linda". Preferiría que se enviaran solos.

Dinero. Recibe pagos en efectivo, por QR (Nequi) y por transferencia, y quiere diferenciarlos. También quiere registrar gastos (esmaltes, pestañas, insumos) y saber al final del periodo el total ganado, los servicios más realizados y los días más llenos.

Prioridades. Cuando se le preguntó qué haría si el programa solo pudiera tener tres funciones, respondió: la agenda, los mensajes de recordatorio y guardar la información de las clientas. Estas tres funciones guían el alcance del proyecto.

B. Requerimientos funcionales y no funcionales

La Tabla I traduce lo anterior a requerimientos verificables, con su prioridad (A: alta, M: media, B: baja) y su estado en esta entrega.

Algunas solicitudes quedaron registradas como trabajo futuro porque exceden el alcance de un programa de escritorio en esta etapa: que las clientas agenden por su cuenta con aprobación de la dueña, el cobro de abonos para separar la cita, las promociones y las fotos de los trabajos realizados.

IV. DISEÑO DEL SISTEMA

A. Diagrama de casos de uso

La primera versión del diagrama tenía once casos de uso y dos relaciones «include» dibujadas con líneas punteadas. Siguiendo la indicación del profesor, el diagrama se depuró: se dejaron solo los casos de uso más importantes y todas las líneas son continuas. El resultado, en la Fig. 1, tiene una sola actora, Valeria (la propietaria), y seis casos de uso.

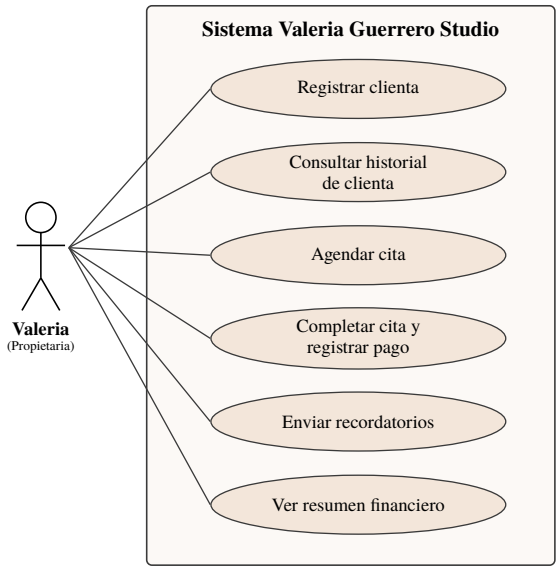


Fig. 1. Diagrama de casos de uso depurado. Todas las asociaciones entre la actora y los casos de uso se dibujan con líneas continuas.

Los criterios de depuración fueron dos. Primero, conservar las tres prioridades que la dueña expresó (agenda, recordatorios y datos de clientas) y el control financiero, que es la razón de registrar pagos y gastos. Segundo, integrar en un mismo caso de uso las acciones que siempre ocurren juntas:

- *Verificar disponibilidad* y *Consultar agenda* pasan a ser parte de *Agendar cita*, ya que el sistema revisa la agenda y los cruces cada vez que se agenda.
- *Registrar pago* se une a *Completar cita*, porque el cobro ocurre en el momento en que se completa el servicio.
- *Cancelar cita* y *Marcar inasistencia* son cambios de estado de una cita y *Gestionar catálogo* es una tarea de configuración; siguen existiendo en el código, pero no son funciones principales para la usuaria.
- *Registrar gasto* alimenta el resumen financiero y se describe dentro de *Ver resumen financiero*.

La Tabla II resume qué hace cada caso de uso y qué método del código lo implementa.

B. Diagrama de clases

La Fig. 2 presenta el diagrama de clases UML [2] del sistema. Para facilitar la lectura se muestran los atributos y los métodos principales; los getters y setters de cada atributo se omiten, aunque están implementados en todas las clases.

Las relaciones del diagrama son las siguientes:

- *Agregación Negocio–demás clases.* `Negocio` guarda arreglos de `Cliente`, `TratamientoEstetico`, `Cita` y `Pago`, pero esos objetos pueden existir por separado (por ejemplo, en `main` se crea la clienta antes de agregarla al negocio).
- *Asociación Cita–Cliente (1).* Cada cita pertenece a una sola clienta; al completarse, la cita llama a

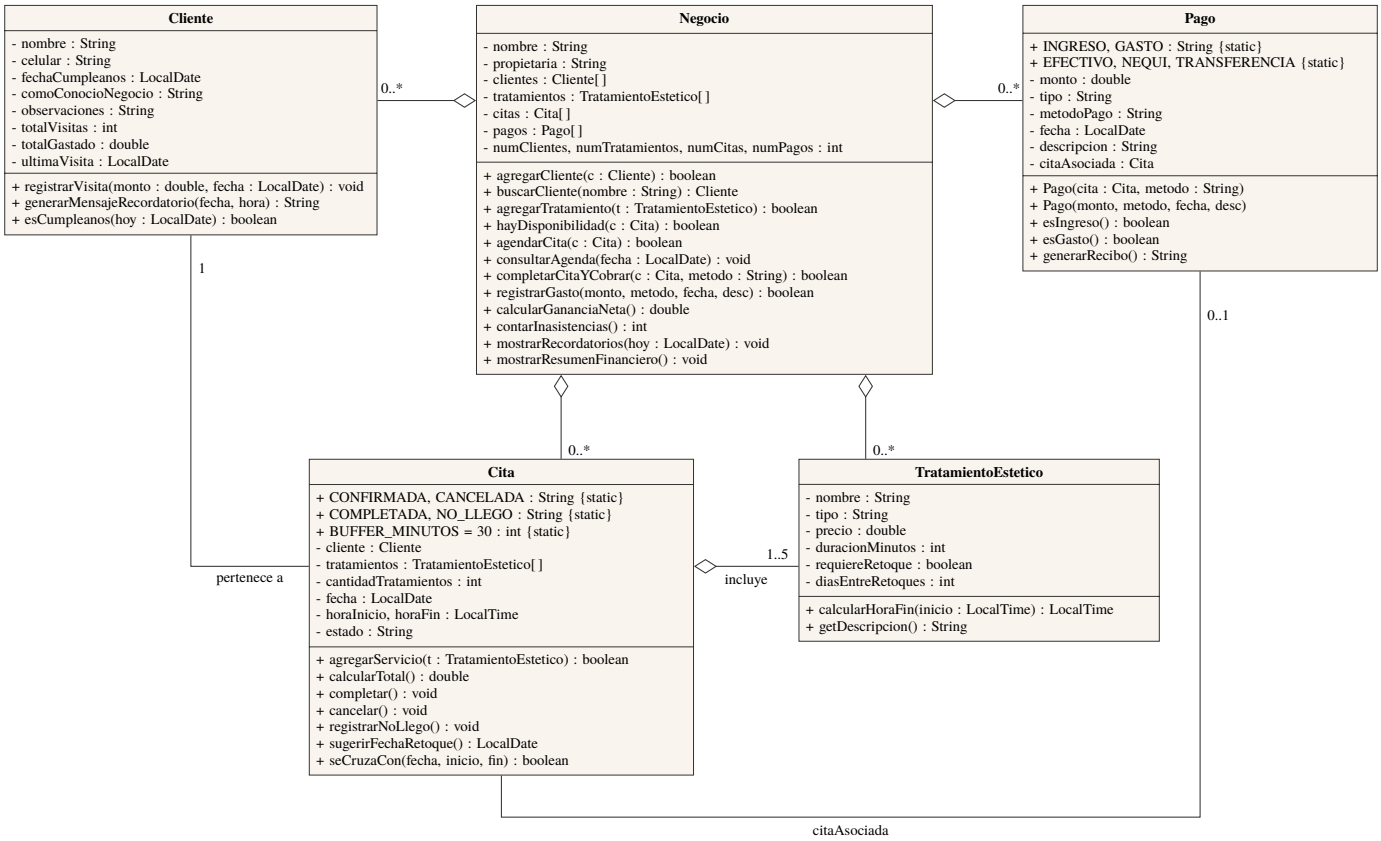


Fig. 2. Diagrama de clases del sistema (se omiten getters y setters). Los rombos vacíos indican agregación: `Negocio` contiene clientas, tratamientos, citas y pagos, y cada `Cita` incluye de uno a cinco tratamientos. Las líneas simples son asociaciones; la de `Pago` con `Cita` es opcional (0..1) porque un gasto no tiene cita asociada.

TABLA II
CASOS DE USO Y MÉTODO DE LA CLASE NEGOCIO QUE LOS IMPLEMENTA

Caso de uso	Descripción	Método principal
Registrar clienta	Crea el perfil con sus datos y contadores en cero	`agregarCliente()`
Consultar historial	Muestra visitas, total gastado, última visita y observaciones	`buscarCliente()`
Agendar cita	Asocia clienta y tratamientos, calcula hora de fin y verifica cruces	`agendarCita()`
Completar cita y registrar pago	Marca la cita como completada, actualiza el historial y guarda el ingreso	`completarCitaYCobrar()`
Enviar recordatorios	Lista los mensajes de las citas confirmadas de hoy y mañana	`mostrarRecordatorios()`
Ver resumen financiero	Ingresos, gastos, ganancia neta, clientas, citas e inasistencias	`mostrarResumenFinanciero()`

`cliente.registrarVisita()`.

- *Agregación Cita–TratamientoEstetico (1..5).* Una cita incluye uno o más tratamientos del catálogo, con un máximo de cinco (`MAX_TRATAMIENTOS`).
- *Asociación Pago–Cita (0..1).* Un ingreso apunta a la cita que lo generó; en un gasto el atributo `citaAsociada` es nulo.

V. DESCRIPCIÓN DE LAS CLASES

Todas las clases siguen la misma estructura, que se presentó en las diapositivas y se ilustra en la Fig. 3 con la clase `Cliente`: (a) atributos privados; (b) métodos que describen lo que hace el objeto; (c) un constructor vacío que inicializa los valores en cero o vacíos; (d) un constructor con parámetros que crea el objeto con todos sus datos, y (e) getters y setters para leer y cambiar cada atributo. Así se cumple el principio de encapsulamiento: los datos solo se modifican a través de los métodos de la propia clase.

A. Cliente

Representa a cada clienta. Guarda los datos que pidió la dueña (nombre, celular, cumpleaños, cómo conoció el negocio y observaciones) y su historial: total de visitas, total gastado y fecha de la última visita. El método `registrarVisita()` actualiza ese historial automáticamente cada vez que se completa una cita. El método `generarMensajeRecordatorio()` construye el texto en el tono solicitado: “Hola linda [nombre], te recordamos que tienes tu cita en Valeria Guerrero Studio el [fecha] a las [hora]. ¡Te esperamos con todo el amor!”. Adicionalmente, `esCumpleanos()` permite saber si la clienta cumple años en una fecha dada.

B. TratamientoEstetico

Modela un servicio del catálogo (por ejemplo, extensiones Tecno clásicas, laminado de cejas o semipermanente) con nombre, tipo (pestañas, cejas, uñas o rostro), precio y duración en minutos. Tiene tres constructores: vacío, con los datos básicos

```java
public class Cliente {
  // a. atributos privados
  private String nombre;
  private int totalVisitas;
  private double totalGastado;
  // c. constructor vacío
  public Cliente() { ... }
  // d. constructor con parámetros
  public Cliente(String nombre, ...) { ... }
  // b. método
  public void registrarVisita(double monto,
                   LocalDate fecha) {
    totalVisitas++; totalGastado += monto;
    ultimaVisita = fecha;
  }
  // e. getter y setter
  public String getNombre() { return nombre; }
  public void setNombre(String n) { ... }
}
```

Fig. 3. Estructura común de las clases, ilustrada con `Cliente`.

y uno adicional para los servicios que requieren retoque, que recibe los días entre retoques (15 para pestañas). El método `calcularHoraFin()` suma la duración a una hora de inicio, y `getDescripcion()` devuelve un resumen con precio, duración y retoque. En el programa se cargan 16 tratamientos reales del catálogo del estudio.

C. Cita

Es la clase que une a una clienta con sus tratamientos en una fecha y hora, representadas con `LocalDate` y `LocalTime` de `java.time` [3]. Su estado se maneja con cuatro constantes estáticas: `CONFIRMADA`, `CANCELADA`, `COMPLETADA` y `NO_LLEGO`. Al agregar un servicio con `agregarServicio()` se recalcula la hora de fin sumando la duración de cada tratamiento más el margen de 30 minutos (`BUFFER_MINUTOS`) que la dueña deja entre clientas:

$$h_{\text{fin}} = h_{\text{inicio}} + \sum_{i=1}^{n} d_i + 30 \text{ min} \tag{1}$$

donde d_i es la duración del tratamiento i. El método `seCruzaCon()` detecta si dos citas del mismo día se solapan, es decir, si $\text{inicio}_A < \text{fin}_B$ y $\text{inicio}_B < \text{fin}_A$; las citas canceladas no bloquean la agenda. `calcularTotal()` suma los precios; `completar()` cambia el estado y actualiza el historial de la clienta, y `sugerirFechaRetoque()` propone la fecha del siguiente retoque si algún servicio lo requiere.

D. Pago

Registra cualquier movimiento de dinero. Tiene dos constructores con parámetros que reflejan los dos casos reales: `Pago(cita, metodo)` crea un ingreso cuyo monto se toma del total de la cita, y `Pago(monto, metodo, fecha, descripcion)` crea un gasto sin cita asociada. El método de pago se guarda como `EFECTIVO`, `NEQUI/QR` o `TRANSFERENCIA`, lo que permite a la dueña diferenciar sus ingresos. `esIngreso()` y `esGasto()` facilitan los cálculos del resumen y `generarRecibo()` produce un comprobante en texto.

```
?`Se agendó cita 2 a las 10:00? false
?`Se agendó cita 2 a las 14:00? true
---- Agenda del 2026-10-23 ----
09:00-12:15 | Laura | Tecno clásicas + Diseño de
  cejas en henna | $150,000 | CONFIRMADA
14:00-16:00 | Camila | Semipermanente | $50,000
---- Recordatorios pendientes ----
Para 3001234567:
  Hola linda Laura, te recordamos que tienes tu
  cita en Valeria Guerrero Studio el 23/10/2026
  a las 09:00 AM. !`Te esperamos con todo el amor!
Sugerir retoque a Laura para el 2026-11-07
Perfil: Laura | Visitas: 1 | Total gastado:
  $150,000 | Obs: Piel sensible
====== RESUMEN VALERIA GUERRERO STUDIO ======
Total ingresos:   $150,000
Total gastos:     $35,000
Ganancia neta:    $115,000
Clientas: 2   Citas totales: 3   Inasistencias: 1
```

Fig. 4. Fragmento de la salida en consola del método `main`.

E. Negocio

Es la clase central del sistema: guarda los arreglos de clientas, tratamientos, citas y pagos, con un contador para cada uno, y ofrece los métodos que usa la propietaria. Antes de guardar una cita, `agendarCita()` llama a `hayDisponibilidad()`, que recorre las citas existentes y usa `seCruzaCon()`; si hay cruce, la cita no se guarda. `completarCitaYCobrar()` completa la cita y registra el ingreso en una sola operación. `mostrarRecordatorios()` filtra las citas confirmadas de hoy y mañana, y `mostrarResumenFinanciero()` consolida ingresos, gastos, ganancia neta, número de clientas, citas e inasistencias.

VI. Prueba del programa

El método `main` de la clase `ValeriaStudio` simula un día de trabajo (23 de octubre de 2026). Primero carga el catálogo y registra dos clientas: Laura, creada con el constructor con parámetros, y Camila, creada con el constructor vacío y los setters. Luego ejecuta los siguientes pasos:

1) Agenda a Laura a las 9:00 con extensiones Tecno clásicas (120 min) y diseño de cejas en henna (45 min). La hora de fin calculada es $9{:}00 + 165 + 30 = 12{:}15$.
2) Intenta agendar a Camila a las 10:00: el sistema la *rechaza* porque se cruza con la cita de Laura. Al moverla a las 14:00, se acepta.
3) Genera los recordatorios de hoy y mañana, completa la cita de Laura con pago por Nequi/QR, marca que Camila no llegó y registra un gasto de $35 000 en esmaltes.

La Fig. 4 muestra la salida real obtenida en consola.

Los resultados son coherentes con lo esperado: la validación de cruces funciona, el historial de Laura se actualiza a una visita y $150 000, la fecha de retoque sugerida es 15 días después (7 de noviembre) y el resumen calcula correctamente la ganancia neta ($150\,000 - 35\,000 = 115\,000$) y una inasistencia.

VII. Estado de avance y trabajo futuro

A la fecha están implementadas y probadas las cinco clases del modelo y los seis casos de uso esenciales funcionan desde `main`. Para la siguiente entrega se plantea:

- Construir el menú principal para que la propietaria use el programa sin modificar el código.
- Mostrar la agenda por semana, como la pidió la dueña, y validar el horario de atención y los días bloqueados.
- Completar el resumen con los servicios más realizados y los días con mayor ocupación.
- Guardar en el perfil de la clienta la lista de servicios realizados, además de los contadores.

VIII. Conclusiones

- Se cumplió el objetivo de diseño: las cinco clases (`Cliente`, `TratamientoEstetico`, `Cita`, `Pago` y `Negocio`) representan directamente los elementos del negocio descritos en la entrevista, y cada una aplica encapsulamiento con atributos privados, dos o más constructores, getters y setters.
- El agendamiento resuelve el principal problema de la dueña, no saber cuánto dura cada cita: el sistema calcula la hora de fin con la suma de duraciones más 30 minutos y rechaza las citas que se cruzan, como se comprobó en la prueba.
- El historial de cada clienta (visitas, total gastado y última visita) se actualiza solo al completar una cita, sin que la propietaria tenga que anotarlo.
- Los recordatorios ya se generan con el mensaje en el tono que pidió la dueña para las citas de hoy y mañana; el envío automático por WhatsApp queda como mejora futura.
- Separar ingresos y gastos en la clase `Pago`, con el método de pago diferenciado, permite calcular la ganancia neta real del negocio y no solo lo que entra.
- Depurar el diagrama de casos de uso a seis casos con líneas continuas dejó un modelo más claro y alineado con las tres prioridades de la clienta: agenda, recordatorios y datos de las clientas.

Referencias

[1] V. Guerrero, "Entrevista sobre los requerimientos del sistema para Valeria Guerrero Studio," comunicación personal, Bogotá, Colombia, 2026.

[2] Object Management Group, *OMG Unified Modeling Language (OMG UML)*, versión 2.5.1, dic. 2017. [En línea]. Disponible: https://www.omg.org/spec/UML/2.5.1

[3] Oracle, "Package java.time," *Java Platform, Standard Edition API Specification*. [En línea]. Disponible: https://docs.oracle.com/en/java/javase/

[4] I. Sommerville, *Ingeniería de software*, 9.ª ed. México: Pearson Educación, 2011.